

가천대학교 스마트시티학과 – 창업프로젝트

Lecture 03

User Interviews and Django Fundamentals

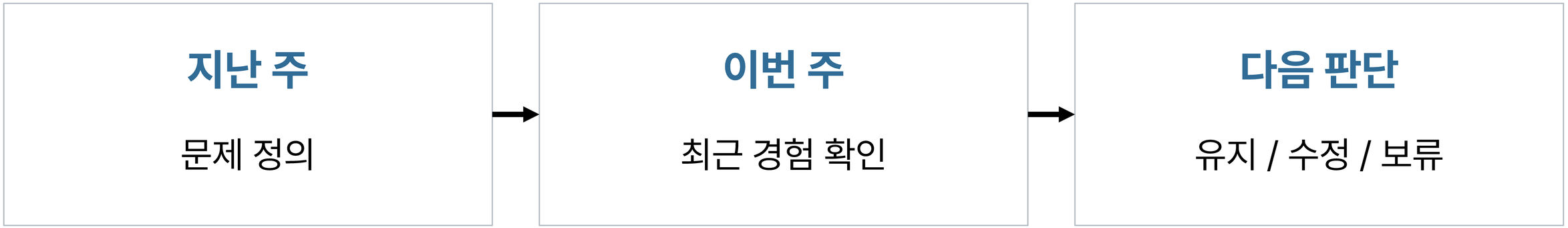
Chris Ohk

utilForever@gmail.com

학습 목표

- 사용자 인터뷰를 통해 처음으로 만들 기능을 정한다.
- Django로 웹 사이트를 만드는 방법을 익힌다.

문제 가설과 사용자 경험



- 문제 문장은 아직 확인하지 않은 내 생각이다.
- 최근 경험을 들어 실제로 어디에서 막혔는지 찾는다.
- 들은 내용에 따라 문제를 유지하거나 고친다.

새로운 문제 정의



모임 담당자

내일 오후에 여섯 명이 함께
이야기할 곳을 정해야 해.

구성원

중앙도서관 스터디룸이면
모두 들어갈 수 있지 않을까?

모임 담당자

정원은 맞는데 그 시간에
이용할 수 있는지 아직 모르겠어.

- 친한 사이인지보다 관련 경험이 있는지를 먼저 확인한다.

어디에서 막혔을까

처음 생각

6명 모일 장소를 찾기 어렵다.

검색 화면이 있으면 해결될까?

확인할 경험

필요한 정보를 못 찾았는가?

찾은 장소를 이용하지 못했는가?

- 장소를 못 찾은 이유와 찾고도 못 쓴 이유는 다르다.
- 어느 단계에서 무엇이 안 됐는지 따로 묻는다.

누구에게 물어볼까



대상 조건

최근 한 달 안에
모임 장소를 직접 찾은 사람

확인할 경험

어떤 정보를 찾았고
어떻게 장소를 정했는가?

- 친한 사이인지보다 관련 경험이 있는지를 먼저 확인한다.

경험을 기준으로 모집



- 대상 조건에는 한 행동과 그 시점을 적는다.
- 불편했던 사람만 모집하면 잘 해결한 경험을 놓친다.
- 성공한 경우와 실패한 경우를 함께 살핀다.

정원만 알면 충분할까

가상 시설	정원	더 확인할 정보
중앙도서관 스터디룸	6명	시간 / 출입 조건
청년센터 모임방	8명	시간 / 출입 조건
학생회관 세미나실	12명	시간 / 출입 조건

- 공간이음은 모임 장소를 비교하는 개인 프로젝트 예시다.
- 정원이 충분해도 원하는 시간에 이용하지 못할 수 있다.

답을 넣지 않는 질문

유도하는 질문

“시설 정보가 흩어져서 불편했죠?”



경험을 여는 질문

“최근 장소는 어떻게 알아보셨나요?”



- 내 예상과 다른 경험도 말할 수 있는 질문을 만든다.

쓴다는 말과 실제 사용

피할 질문	이런 앱이 있으면 매번 쓰실 건가요?
부족한 이유	미래의 약속만으로 실제 사용을 알 수 없다.
바꾼 질문	최근 모임 장소를 정할 때 무엇을 사용하셨나요?

- “쓸 것 같다”는 답은 아직 일어나지 않은 행동이다.
- 최근 비슷한 일을 어떻게 해결했는지 먼저 묻는다.

최근 한 번을 자세히



처음에 묻기

"가장 최근 모임 장소를
어떻게 알아보셨나요?"

뒤이어 확인

"몇 명이 모였나요?"
"어떤 정보를 확인했나요?"

- 한 번의 경험을 고른 뒤 행동과 조건을 구체적으로 묻는다.

후속 질문의 역할

내 해석을 넣은 질문

불편해서 다른 곳으로 바꿨군요?

결국 통합 앱이 필요한 거네요?

실제 장면을 묻는 질문

그다음에 무엇을 하셨나요?

“못 썼다”는 상황을 설명해 주실래요?

- 후속 질문은 방금 들은 말의 뜻을 더 자세히 묻는 질문이다.
- 그다음 행동과 선택을 바꾼 이유를 확인한다.

경험을 시간순으로 정리



1. 안내 찾기



2. 조건 비교



3. 이용 여부 확인

- 장소를 찾은 과정과 실제 이용 결과를 나누어 묻는다.

서비스 설명보다 경험 듣기

창업프로젝트
User Interviews

시작 안내	최근 모임 장소를 찾은 경험을 10분 정도 듣고 싶습니다.
첫 질문	가장 최근에 장소를 알아본 때는 언제인가요?
덧붙일 안내	잘 해결된 경험도 도움이 됩니다.

- 기능부터 소개하면 대화가 아이디어 평가로 흐르기 쉽다.
- 대화의 목적과 시간을 알린 뒤 최근 경험을 묻는다.

A는 정보를 모아 비교했다



수업용 가상 인터뷰 A

지난 금요일 6명 모임 장소를 찾으려고
시설 홈페이지 세 곳을 돌아봤어요.
정원과 이용 시간이 달라 비교표를 만들었죠.

후속 질문

비교표에는 무엇을 적었고,
어디에서 찾으셨나요?

- 비교표를 만든 행동은 확인되지만 앱 사용 의향은 모른다.

B는 출입 조건에 막혔다



수업용 가상 인터뷰 B

정원과 위치는 찾았어요.
그런데 18시 이후에는 외부인이
들어갈 수 없어서 그 방을 쓰지 못했어요.

후속 질문

18시 이후 출입 조건은 언제 알게 되셨나요?

- 조건을 안내해도 시설의 출입 규칙 자체는 바뀌지 않는다.

C는 이미 해결했다



수업용 가상 인터뷰 C

우리 모임은 늘 쓰던 방이 있어요.
메신저로 담당자에게 물으면 끝나서
새 검색 서비스는 필요하지 않았어요.

후속 질문

최근에는 담당자에게 무엇을 물어
장소를 정했나요?

- 문제가 없었던 경험도 서비스가 필요한 범위를 알려 준다.

같은 모임 / 다른 경험

가상 인터뷰	들은 내용	아직 모르는 것
A	홈페이지 3곳의 조건 비교	비교에 든 노력
B	18시 이후 외부인 출입 불가	조건을 안 시점
C	기존 방 / 메신저로 해결	방법이 잘된 이유

- 모임을 준비해도 막힌 단계와 해결 방법은 서로 다르다.
- A의 경험을 B와 C에게 그대로 적용하지 않는다.

정보 안내와 규칙 변경





화면에서 조건 비교

정원 / 시간 / 조건을 같은 형식으로 안내한다.

현장의 출입과 이용 규칙

시설 담당자가 이용 자격과 실제 상태를 확인한다.

- 조건을 안내하는 기능과 실제 이용을 허용하는 권한은 다르다.

이미 잘되는 방법도 살피기



- 반례는 예상한 문제가 없거나 다른 방법으로 해결한 경험이다.
- 기존 대안에는 다른 앱뿐 아니라 전화와 지인도 포함된다.
- 기존 방법으로 충분한 조건을 알면 대상 범위를 좁힐 수 있다.

인터뷰 결과의 해석 범위

지나치게 넓은 결론

학생의 33%가 통합 검색을 원한다

세 명을 만나 수요를 확인했다

지금 쓸 수 있는 결론

A는 여러 시설의 조건을 비교했다

비슷한 경험이 더 있는지 확인한다

- 소수의 인터뷰는 경험의 차이와 다음 질문을 찾는 데 쓴다.
- 세 명 중 한 명의 경험을 전체 학생의 33%로 바꿀 수 없다.

다음에 확인할 질문

대상	다음 질문	알고 싶은 것
A	비교표에 무엇을 적었나요?	필요한 비교 정보
B	출입 조건은 언제 알았나요?	정보 부족 / 운영 제약
C	최근에는 어떻게 방을 정했나요?	기존 방법이 잘된 조건

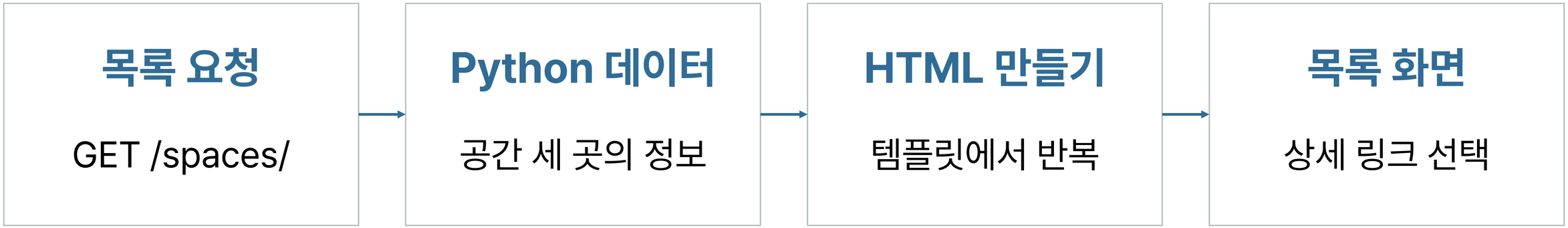
- 다음 질문은 현재의 판단을 바꿀 수 있는 정보를 찾는다.
- 서로 다른 경험에는 서로 다른 후속 질문이 필요하다.

인터뷰 질문과 화면 정보

사용자의 질문	데이터 후보	보여 줄 화면
6명이 들어갈 수 있나?	시설명 / 정원	목록에서 비교
언제 이용할 수 있나?	이용 시간 / 출입 조건	상세에서 확인
조건을 어디서 다시 확인하나?	안내 출처 / 확인 방법	상세에서 안내

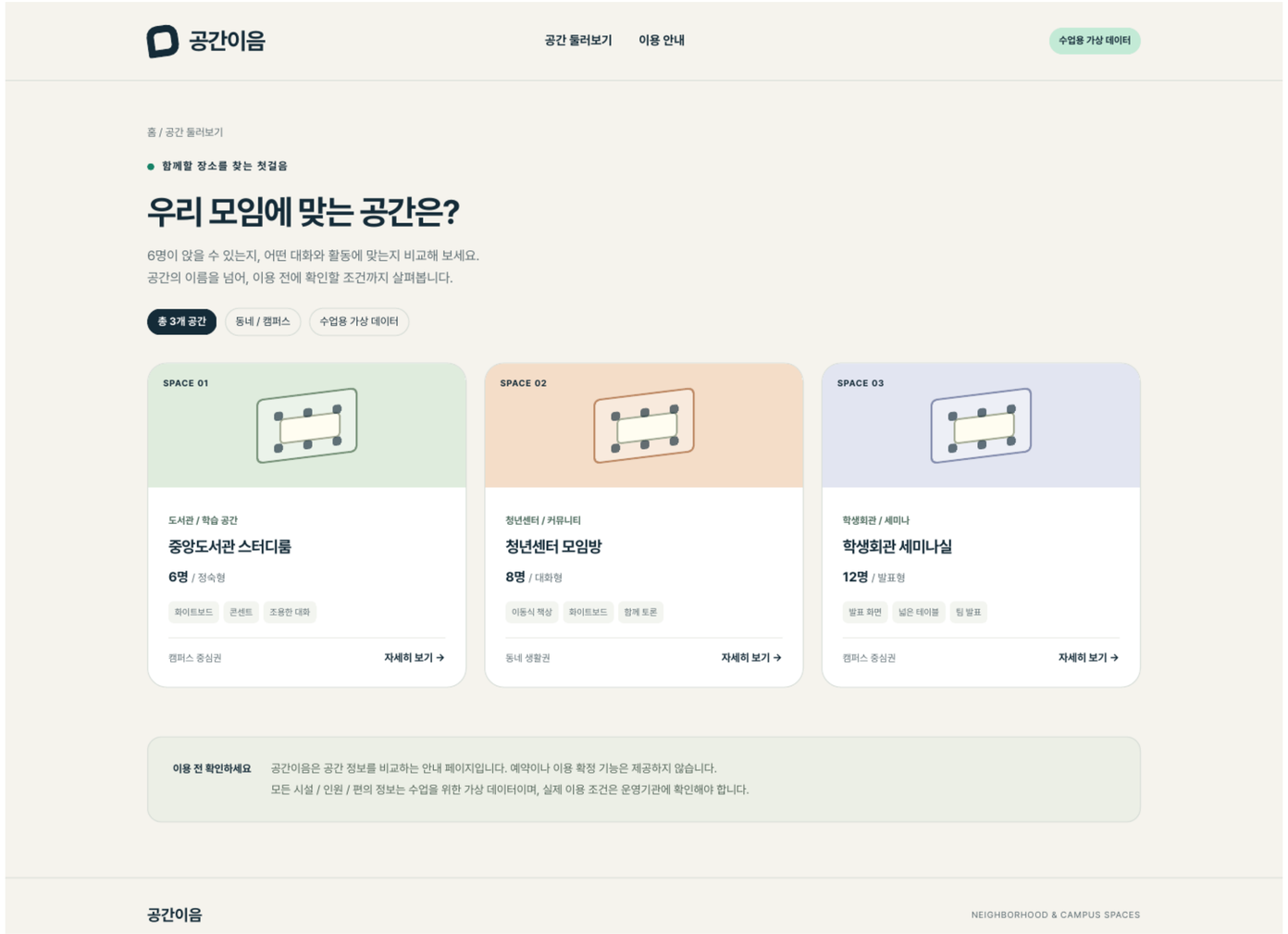
- 사용자가 판단할 때 필요했던 정보는 화면에 담을 후보가 된다.
- Django 실습에서는 가상 시설 세 곳의 목록과 상세를 만든다.

오늘 만드는 웹 서비스



- Python 리스트에 저장한 공간 정보를 HTML 화면에 표시한다.
- 목록에서 공간을 선택하면 해당 공간의 상세 화면을 연다.

완성 화면 먼저 살펴보기

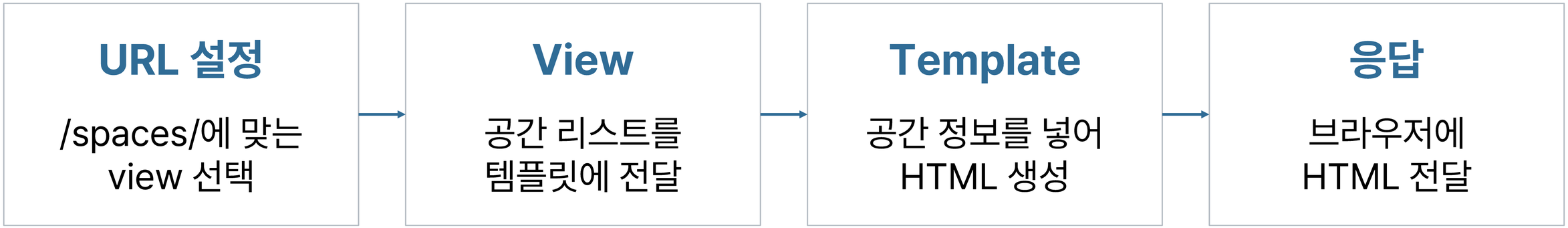

공간이음
공간 둘러보기
이용 안내
수업용 가상 데이터
홈 / 공간 둘러보기
함께할 장소를 찾는 첫걸음
우리 모임에 맞는 공간은?
6명이 앉을 수 있는지, 어떤 대화와 활동에 맞는지 비교해 보세요.
공간의 이름을 넘어, 이용 전에 확인할 조건까지 살펴봅니다.
총 3개 공간
동네 / 캠퍼스
수업용 가상 데이터
SPACE 01
도서관 / 학습 공간
중앙도서관 스터디룸
6명 / 정숙형
화이트보드
콘센트
조용한 대화
캠퍼스 중심권
자세히 보기 →
SPACE 02
청년센터 / 커뮤니티
청년센터 모임방
8명 / 대화형
이동식 책상
화이트보드
함께 토론
동네 생활권
자세히 보기 →
SPACE 03
학생회관 / 세미나
학생회관 세미나실
12명 / 발표형
발표 화면
넓은 테이블
팀 발표
캠퍼스 중심권
자세히 보기 →
이용 전 확인하세요
공간이음은 공간 정보를 비교하는 안내 페이지입니다. 예약이나 이용 확정 기능은 제공하지 않습니다.
모든 시설 / 인원 / 편의 정보는 수업을 위한 가상 데이터이며, 실제 이용 조건은 운영기관에 확인해야 합니다.
공간이음
NEIGHBORHOOD & CAMPUS SPACES

Django는 무엇인가?

개발자가 정할 내용	Django로 구현할 부분
어떤 공간 정보를 보여 줄까?	Python view에서 데이터 준비
어떤 주소로 접속하게 할까?	URL 설정으로 view 연결
화면에 정보를 어떻게 배치할까?	템플릿으로 HTML 응답 생성

- Django는 Python으로 웹 서비스를 만드는 데 쓰는 프레임워크다.
- 개발자는 Django의 기능을 이용해 URL과 화면을 연결할 수 있다.

요청이 화면이 되는 과정



- URL 설정은 요청 경로를 확인하고 실행할 view를 선택한다.
- view가 준비한 데이터를 템플릿에 넣어 HTML 응답을 만든다.

프로젝트와 app의 역할

프로젝트 설정 / config

서비스 전체의 설정을 보관한다.

각 앱의 URL 설정을 연결한다.

공간 안내 앱 / spaces

공간 목록과 상세 기능을 만든다.

URL 설정과 view를 포함한다.

- 프로젝트는 서비스 전체를 구성하며 앱은 기능별 코드를 모은다.
- 이번 프로젝트는 spaces 앱 하나로 공간 안내 기능을 만든다.

어느 파일을 수정할까?

파일	역할	예
config/urls.py	app URL 연결	spaces.urls 포함
spaces/urls.py	경로와 view 연결	/spaces/
spaces/views.py	데이터와 응답 준비	space_list 함수
templates/spaces/	HTML 출력 구조	space_list.html

- 요청 하나를 따라가며 URL 설정 / view / 템플릿 파일을 찾는다.

프로젝트용 가상환경 만들기


```
# Windows
py -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt

# macOS / Linux
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
```

- 가상환경을 만들면 프로젝트별로 Python 패키지를 관리할 수 있다.
- 패키지 설치와 서버 실행에 같은 가상환경의 Python을 사용한다.

개발 서버 실행하기

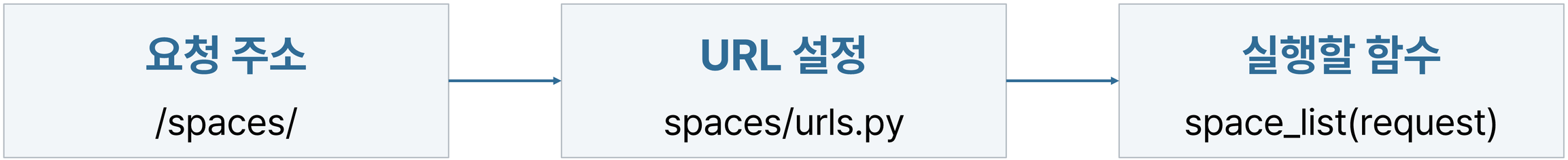

```
# Windows
.venv\Scripts\python manage.py runserver

# macOS / Linux
.venv/bin/python manage.py runserver
```

- manage.py가 있는 폴더에서 명령을 실행하고 접속 주소를 연다.
- runserver는 개발 중 확인에 사용하며 실제 배포에는 쓰지 않는다.

URL을 view에 연결하기


```
# spaces/urls.py / 기존 파일의 두 항목
# TODO-01만 수정하고 나머지 경로는 유지
path("", views.home, name="home"),
path("spaces/", views.space_list, name="space_list"),
```



- Django는 요청 주소와 일치하는 경로를 찾아 view를 호출한다.

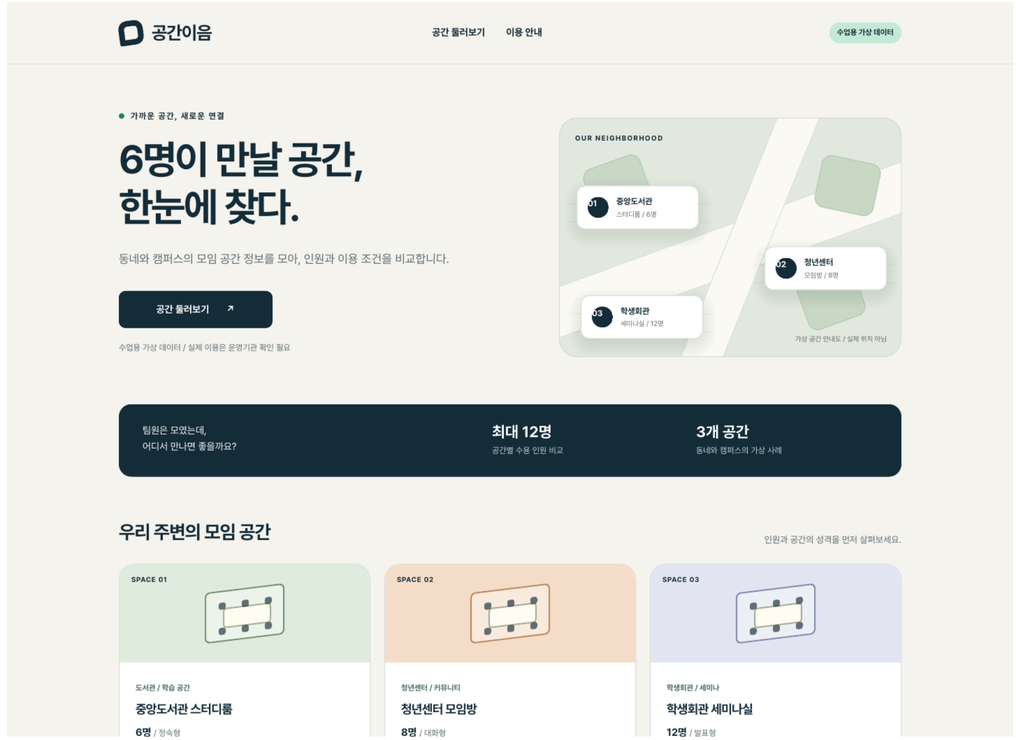
view가 만드는 HTML 응답

spaces/views.py

```
from django.shortcuts import render
from .data import SPACES

def home(request):
    return render(request,
        "spaces/home.html",
        {"spaces": SPACES},
    )
```

홈 페이지의 표시 결과



- render는 템플릿과 데이터를 합쳐 HTML 응답을 만든다.

context로 데이터 전달하기


```
# Python 데이터
space = {"name": "청년센터 모임방",
         "capacity": 8}
context = {"space": space}

# Django 템플릿(일부)
<h3>{{ space.name }}</h3>
<p>{{ space.capacity }}명</p>
```

템플릿이 만든 HTML의 표시 결과



- 같은 템플릿도 전달받은 데이터에 따라 다른 내용을 표시한다.

실습 1 : 목록 경로 연결

목록 주소가 목록 view로 연결되게 한다.

1. TODO-01 : 목록 경로를 views.space_list에 연결한다.
2. /spaces/에 접속해 목록 화면의 제목을 확인한다.
3. 아직 데이터가 연결되지 않아 목록은 비어 있다.

공간 데이터의 구조

키	예시 값	화면에서의 역할
id	1	상세 페이지를 구분
name	중앙도서관 스터디룸	공간 이름
capacity	6	이용 정원
description	공간 설명 문자열	상세 안내

- 딕셔너리 하나에 한 공간의 이름 / 정원 / 설명을 담는다.
- 여러 공간의 딕셔너리를 리스트 SPACES에 넣어 관리한다.

목록 데이터 전달

view에서 전달

```
# spaces/views.py / 기존 함수만 수정
def space_list(request):
    context = {"spaces": SPACES}
    return render(request,
        "spaces/space_list.html", context)
```

템플릿에서 사용

```
{% for space in spaces %}
  {{ space.name }}
{% endfor %}
```

context의 키 이름과
템플릿의 변수 이름이 같다.

- "spaces"라는 이름으로 전달한 목록을 템플릿에서 꺼내 쓴다.

템플릿의 반복문

Django 템플릿

```
{% for space in spaces %}
  <h3>{{ space.name }}</h3>
  <p>정원 {{ space.capacity }}명</p>
{% empty %}
  <p>표시할 공간이 없습니다.</p>
{% endfor %}
```

반복하며 선택하는 공간

1 중앙도서관 스터디룸 / 6명

2 청년센터 모임방 / 8명

3 학생회관 세미나실 / 12명

- for는 목록에서 공간 하나를 꺼내 같은 HTML 구조를 반복한다.

실습 2 : 공간 목록 출력

Python 리스트의 세 공간을 목록에 표시한다.

1. TODO-02 : SPACES를 spaces라는 이름으로 전달한다.
2. TODO-03 / 04 : 반복문과 공간 이름을 작성한다.
3. 화면의 이름과 정원을 data.py와 대조한다.

데이터와 화면 대조하기



- 리스트의 데이터 세 건이 같은 구조의 공간 카드 세 개로 표시된다.

상세 주소와 공간 id


```
# spaces/urls.py — 이미 제공된 경로
path("spaces/<int:space_id>/",
     views.space_detail, name="space_detail")

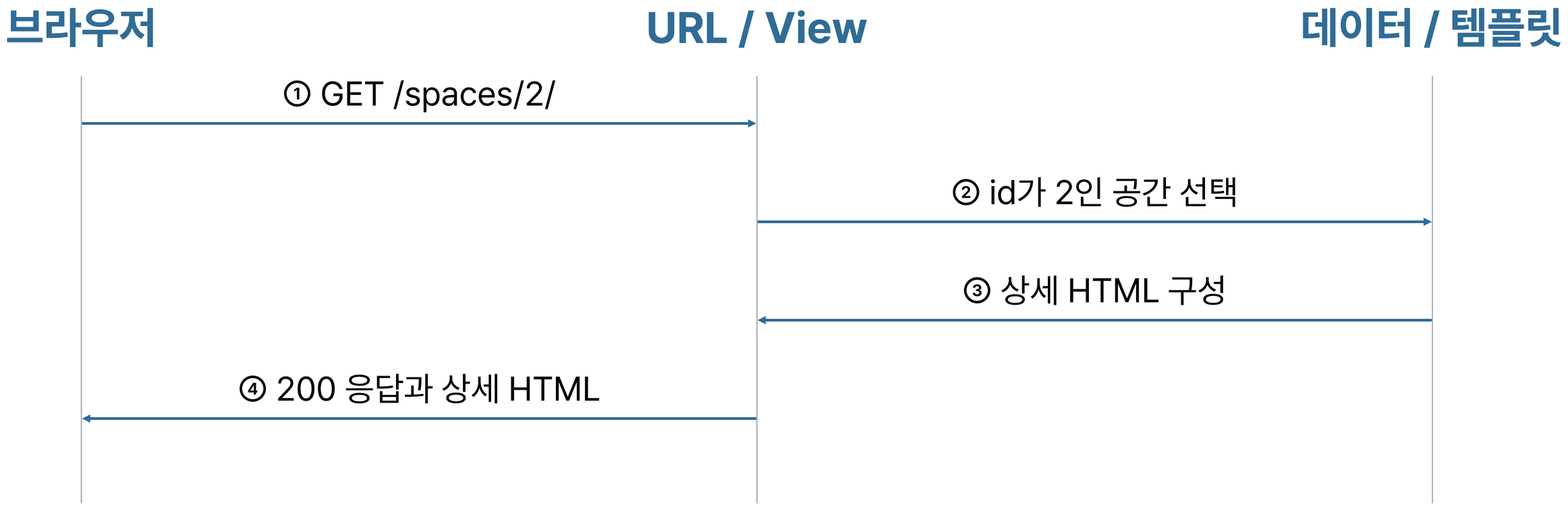
# /spaces/2/ 요청 → space_detail(request, space_id=2)
# views.py의 제공된 함수에서 id가 2인 데이터를 선택
```

/spaces/2/ ⟶ **space_id = 2**

id가 2인 공간을 찾는다.

- 주소의 숫자는 목록 순서가 아니라 공간 데이터의 id다.

상세 요청을 순서대로 따라가기



- 요청한 id의 공간을 찾으면 상세 HTML과 200 응답을 보낸다.
- 해당 id가 없으면 안내 화면과 404 응답을 보낸다.

공간마다 다른 상세 링크 만들기


```
<a href="{% url 'spaces:space_detail' space.id %}">
  자세히 보기
</a>
```

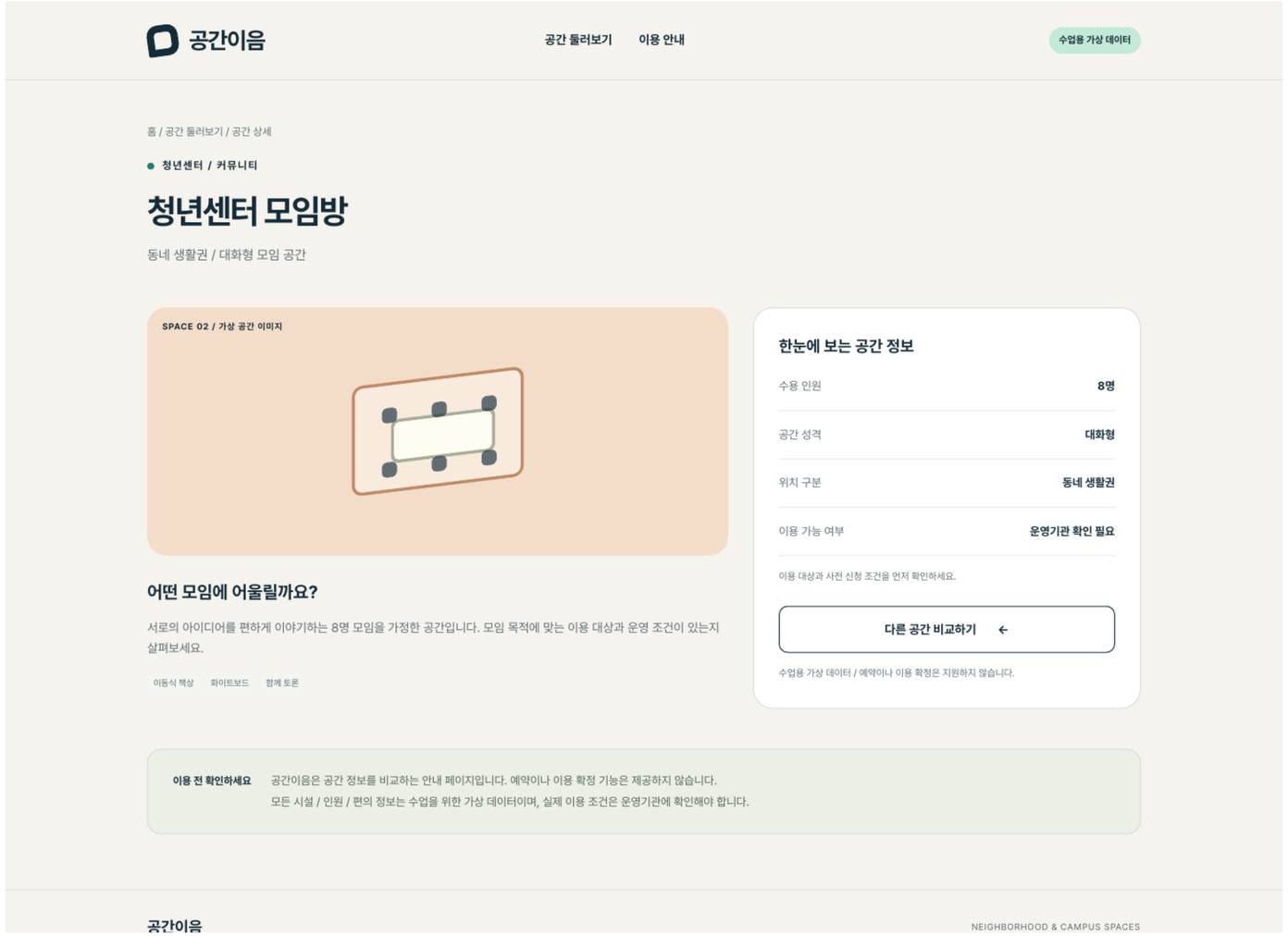
- {% url %}은 URL 이름과 공간 id를 이용해 주소를 만든다.
- space.id가 2이면 상세 링크의 주소는 /spaces/2/가 된다.

실습 3 / 목록과 상세 연결

선택한 공간의 상세 화면으로 이동하게 한다.

1. TODO-05 : 현재 공간의 id로 상세 링크를 만든다.
2. TODO-06 : 상세 화면에 공간 이름과 정원을 표시한다.
3. 세 공간의 링크와 /spaces/999/ 응답을 확인한다.

상세 화면의 정보 확인하기



브라우저에는 무엇이 전달될까?

서버에서 실행하는 코드

```
# views.py
context = {"spaces": SPACES}

# space_list.html(일부)
{% for space in spaces %}
  <h3>{{ space.name }}</h3>
  <p>{{ space.capacity }}명</p>
{% endfor %}
```

브라우저가 받은 HTML

```
<!-- 한 카드의 핵심 내용 -->
<h3>청년센터 모임방</h3>
<p>8명</p>
```



- 브라우저에는 Python 코드가 아니라 완성된 HTML을 전달한다.

오류 화면의 확인 순서



404

요청한 id의 공간이 없다.

500

템플릿 경로 등 실행 오류를 살핀다.

200 / 일부 빈칸

변수 이름의 오타를 확인한다.

- 상태 코드와 화면에 빠진 내용을 함께 확인한다.

감사합니다.

utilForever@gmail.com

https://github.com/utilForever

Facebook, Twitter, Instagram, LinkedIn: utilForever